Algebrite - Template

Template for the Algebrite JavaScript Computer-Algebra-System (CAS) https://algebrite.org to be used in LiaScript. It makes Markdown code-blocks executable, renders results as pretty formulas and lets you check quiz answers mathematically, so `0,5`, `1/2`, `50%` and `\frac{1}{2}` are all accepted as the same answer.

Try it on LiaScript:

https://liascript.github.io/course/?https://raw.githubusercontent.com/liaTemplates/algebrite/master/README.md

See the project on Github:

https://github.com/liaTemplates/algebrite

Like with other LiaScript templates, there are three ways to integrate Algebrite, but the easiest way is to import it.

1. Load the latest macros via (this might cause breaking changes)

   ```
   import: https://raw.githubusercontent.com/liaTemplates/algebrite/master/README.md
   ```

   or the current version 0.7.1 via:

   ```
   import: https://raw.githubusercontent.com/LiaTemplates/algebrite/0.7.1/README.md
   ```

2. **Copy the definitions into your Project**, see Sec. Implementation
3. Clone this repository on GitHub

Overview of all macros:

Macro	Purpose
`@Algebrite.eval`	run a code-block and show the plain result
`@Algebrite.pretty`	run every line of a code-block and show it as a numbered formula
`@Algebrite.repl`	run a code-block, then keep a terminal open for further input
`@Algebrite.check(solution)`	quiz: answer must be mathematically equal to the solution
`@Algebrite.check2(solution, tolerance)`	quiz: answer may differ from the solution by the tolerance
`@Algebrite.check_margin(min, max)`	quiz: answer must lie within `min` and `max`
`@Algebrite.check_expression(equation)`	quiz: answer must be an equivalent equation

All `check` macros take an optional last parameter `units=1` or `units=0`, see Sec. Units in Quizzes.

More Information

Algebrite is...

- **lightweight**: made to be simple to comprehend and extend, it only depends on BigInteger.js by Peter Olson.
- **self-contained**: doesn't need connection to servers or another “backend” CAS
- **a library**: beyond use as an interactive tool, Algebrite can be embedded in your applications and extended with custom functions.
- **free**: MIT-Licenced

Function reference: http://algebrite.org/docs/latest-stable/reference.html

`@Algebrite.eval`

Attach `@Algebrite.eval` directly below a code-block to make it executable. Every line is evaluated in order and its result is shown as plain text, everything after `#` or `--` is a comment. These examples are taken from the website http://algebrite.org, double-click onto the listing to edit it.

```
(3 * x - 5x)^3 * (x + x)  # 5x means 5*x

60!                       # factorial with exact big integers
```

```
-16*x^4
832098711274139014427634118322336438075417260636124595244927769640960000000
```

The following example might take a few seconds …

```
f=sin(t)^4-2*cos(t/2)^3*sin(t)  # define f

f=circexp(f)                    # rewrite f in exponential form

defint(f,t,0,2*pi)              # integrate over one period
```

```
-16/5+3/4*pi
```

Pretty Output with `pretty(1)`

By default, `@Algebrite.eval` prints results as plain text. After `pretty(1)`, every following result is rendered as a formula, until `pretty(0)` switches back. The setting applies to the rest of the page, similar to `units(1)`, and it also works in `@Algebrite.repl`. `@Algebrite.pretty` is the same as `@Algebrite.eval` with `pretty(1)` always on.

```
expand((x+1)^3)      # plain text

pretty(1)            # results as formulas from here on

expand((x+1)^3)

sum(1/k,k,1,n)

pretty(0)            # back to plain text
```

```
x^3+3*x^2+3*x+1
```

$$x^3 + 3x^2 + 3x + 1$$

$$\sum_{k=1}^{n} \frac{1}{k}$$

Results and `draw` charts appear in the order of the lines, and errors are shown in red.

`@Algebrite.pretty`

`@Algebrite.pretty` works like `@Algebrite.eval` with `pretty(1)` always on: every result is rendered as a LaTeX formula, numbered by its line. Errors are shown in red.

````
```Maxima
(x + 1)^2 / 2
sqrt(8) * x
integral(x^2, x)
d(sin(x) * x, x)
```
@Algebrite.pretty
````

Result:

```
(x + 1)^2 / 2
sqrt(8) * x
integral(x^2, x)
d(sin(x) * x, x)
```

$$\frac{x^2}{2} + x + \frac{1}{2} \tag{1}$$

$$2\sqrt{2}x \tag{2}$$

$$\frac{x^3}{3} \tag{3}$$

$$sin(x) + xcos(x) \tag{4}$$

`@Algebrite.repl`

`@Algebrite.repl` runs the code block like `@Algebrite.eval`, but then keeps a terminal open. Every line typed into it is evaluated with the same variables, so definitions from the code block can be used interactively. Try `f(2)`, `derivative(f(x),x)`, `pretty(1)` or `draw(f(x), x, -2, 2)`.

```
f(x) = x^3 - 2*x   # define a function to play with

f(1)
```

```
-1
```

Quizzes

All `check` macros are attached below a LiaScript text-input quiz `[[ ... ]]`. Instead of comparing strings, the answer is handed to Algebrite and compared mathematically with the solution. That means, learners do not have to guess the “correct” notation.

What learners can type into a quiz field:

Input style	Examples
Algebrite syntax	`1/3`, `3*4`, `x^2-1`, `sqrt(2)`, `pi/2`
LaTeX syntax	`\frac{1}{3}`, `\sqrt{2}`, `2\cdot x`, `\frac{\pi}{2}`
Decimal comma	`1,5` is read as `1.5`
Percentages	`50%` is read as `0.5`

LaTeX and Algebrite syntax can be used side by side, the template detects LaTeX automatically as soon as the input contains a command like `\frac`, `\sqrt` or `\cdot`, and converts it before the comparison. The solution in the macro parameter itself has to be written in Algebrite syntax.

Notes:

- LaTeX is only detected via commands (`\frac`, `\sqrt`, `\cdot`, …). Plain braces like `x^{2}` without any command are not converted, use `x^2` instead.
- Multiple solutions are passed as a list, separated by semicolons: `[ 2/5; 5/7; 3/4 ]`. Commas are not used as separators, since they are interpreted as decimal commas.
- Empty fields are always counted as wrong.

Units in Quizzes

With units switched on, symbols like `m`, `km`, `s` or `kg` are treated as units, so `1500m` and `1.5km` are the same answer. An answer with a wrong or missing unit, like `1.5` or `1.5kg` instead of `1.5km`, is counted as wrong.

Macro	Units by default	Switch with
`@Algebrite.check2`	on	`units=0`
`@Algebrite.check_margin`	on	`units=0`
`@Algebrite.check`	off	`units=1`
`@Algebrite.check_expression`	off	`units=1`

Units are off for `check` and `check_expression`, because formulas often use variables that are also unit symbols: In `F = m*a`, `F` would be read as farad, `m` as metre and `a` as year, and the correct answer would be rejected. Only switch units on there, if the solution is a quantity like `1.5km`.

`@Algebrite.check`

`@Algebrite.check(solution)` accepts every answer that is mathematically equal to the solution. Try out different results like `12,0`, `3*4`, `\frac{24}{2}` or `1200%`.

```
6 + 6 = [[12]]
@Algebrite.check(12)
```

6 + 6 = ?

The same works with symbolic expressions. Try different forms of `x ^ 2 - 1`, like `-1 + x * x`, `(x+1)*(x-1)` or `\frac{2x^2-2}{2}`.

```
[[x ^ 2 - 1]]
@Algebrite.check(x^2-1)
```

If a quiz contains multiple input fields, pass a list of solutions separated by semicolons, one per field and in the same order. Try `0,4` or `\frac{2}{5}` for the first field.

```
$x=\;$ [[ 2/5 ]] $\;\;\wedge\;\; y=$  [[ 5/7 ]] $\;\;\wedge\;\; z=$  [[3/4
@Algebrite.check([ 2/5; 5/7; 3/4 ])
```

$x =$? $\wedge$ $y =$? $\wedge$ $z =$?

For answers with units, add `units=1`. Try `1500m` or `1,5km`.

```
[[ 1.5km ]]
@Algebrite.check(1.5km, units=1)
```

`@Algebrite.check2`

If a result might be rounded, use `@Algebrite.check2(solution, tolerance)`. An answer is correct if it differs from the solution by at most the tolerance. Try `0,33`, `33%` or `\frac{1}{3}`.

```
[[1/3]]
@Algebrite.check2(1/3, 0.01)
```

For multiple input fields, pass a list of solutions and a list of tolerances. If only one tolerance is given, it is used for all fields.

```
$a=$ [[ 1/3 ]]\
$b=$ [[ 2/3 ]]\
$c=$ [[ 3/3 ]]
@Algebrite.check2([ 1/3 ; 2/3 ; 3/3 ], [0.01 ; 0.01 ; 0.01])
```

$a =$?
$b =$?
$c =$?

Units are switched on by default, so solution and tolerance can have different units. Try `1490m` or `1,51km`.

```
[[ 1.5km ]]
@Algebrite.check2(1.5km, 20m)
```

`@Algebrite.check_margin`

`@Algebrite.check_margin(min, max)` accepts every answer within a range, both bounds included. This is useful for measurements or estimations. Units are switched on by default, try `1,45km`, `1450m` or `1.6km`.

```
-> [[ 1.5km ]]
@Algebrite.check_margin(1.4km, 1.6km)
```

→ ?

`@Algebrite.check_expression`

`@Algebrite.check_expression(equation)` checks equations. Both sides are moved to one side and compared, so `x^2 - 1 = 2x`, `x^2 - 2x - 1 = 0`, `-1 = 2x - x^2` or `x^2 - 1 = 2\cdot x` are all accepted. If no `=` is given, `= 0` is assumed. Equations that were multiplied by a factor, like `2x^2 - 2 = 4x`, are not accepted.

```
[[x ^ 2 - 1 = 2x]]
@Algebrite.check_expression(x^2-1-2x=0)
```

CAS - Tutorial

A guided tour through Algebrite's built-in function library, grouped by topic. Every example below is a live code block using `@Algebrite.pretty` (renders the result as a formula) or `@Algebrite.eval` (renders the result as plain text, or as a formula after `pretty(1)`) — double-click any block to edit it and try your own expressions. A complete alphabetical reference of every function follows at the end.

Comments

Everything after `#` or `--` up to the end of the line is a comment and is ignored by Algebrite. A comment can take up a whole line or follow an expression, and the examples in this tutorial use comments to explain each line. There is no block comment, and `//` is not a comment but a syntax error.

```
1  # a comment on its own line
2  x + x          # a comment after an expression
3  y * y          -- the same with a double minus
```

$$2x \tag{2}$$

$$y^2 \tag{3}$$

Because `--` starts a comment, a double minus cuts off the rest of the line. `5--2` is read as `5`, so write `5-(-2)` instead. This also applies to answers typed into a quiz.

```
1  5--2           # everything after -- is ignored
```

```
2 5-(-2)          # parentheses keep the second minus
```

```
5
7
```

1. Arithmetic & Simplification

Function	Description
`add(a, b, ...)`	Sum of two or more terms
`multiply(a, b, ...)`	Product of two or more factors
`power(a, b)`	`a` raised to the power `b`, same as `a^b`
`simplify(expr)`	General purpose simplification
`rationalize(expr)`	Combines terms over a common denominator
`expand(expr)`	Multiplies out products and powers
`factor(expr)`	Factors a polynomial or integer
`condense(expr)`	Factors common terms out of a sum (the reverse of `expand`)
`float(expr)`	Numeric (floating point) evaluation
`min(a, b, ...)`, `max(a, b, ...)`	Smallest / largest argument
`log(x, b)`, `log10(x)`, `log2(x)`	Logarithm to base `b`, 10 or 2
`sqrt(x)`, `cbrt(x)`, `root(x, n)`	Square, cube and `n`-th root
`apart(f, x)` / `partfrac(f, x)`	Partial fraction decomposition of `f` in `x`

```
1 add(2,3)                # same as 2+3
2
3 multiply(2,x)           # same as 2*x
4
5 power(x,3)              # same as x^3
```

```
6
7  simplify((x^2-1)/(x-1))  # cancels the common factor x-1
8
9  rationalize(1/x+1/y)     # one fraction over x*y
10
11 expand((x+1)^3)          # multiply out the cube
12
13 factor(x^2-1)            # difference of two squares
14
15 condense(x/2+y/2)        # pull out the common factor 1/2
16
17 float(1/3)               # decimal approximation
```

$$5 \tag{1}$$

$$2x \tag{3}$$

$$x^3 \tag{5}$$

$$x + 1 \tag{7}$$

$$\frac{(x + y)}{xy} \tag{9}$$

$$x^3 + 3x^2 + 3x + 1 \tag{11}$$

$$(x - 1)(x + 1) \tag{13}$$

$$\frac{(x + y)}{2} \tag{15}$$

$$0.333333... \tag{17}$$

`min` and `max` stay unevaluated when the order of their arguments cannot be decided. `log(x, b)` is exact when `x` is a power of `b`, and `apart` keeps repeated factors such as `(x-1)^2` in factored form.

```
min(3,1,2)                      # smallest argument

max(pi,3)                       # pi is larger than 3

min(x,1)                        # unevaluated, the value of x is unkn

log(8,2)                        # exact, because 8 = 2^3

```

```
log10(1000)                       # logarithm to base 10

log(x,b)                          # symbolic base

cbrt(27)                          # cube root

root(x,n)                         # n-th root

apart((5*x+1)/((x-1)*(x+2)),x)    # one fraction per linear factor

apart(1/((x-1)^2*(x+2)),x)        # the repeated factor (x-1)^2 stays
  factored
```

$$1 \tag{1}$$

$$\pi \tag{3}$$

$$min(x, 1) \tag{5}$$

$$3 \tag{7}$$

$$3 \tag{9}$$

$$\frac{log(x)}{log(b)} \tag{11}$$

$$3 \tag{13}$$

$$x^{\frac{1}{n}} \tag{15}$$

$$\frac{2}{(x-1)} + \frac{3}{(x+2)} \tag{17}$$

$$\frac{1}{3(x-1)^2} - \frac{1}{9(x-1)} + \frac{1}{9(x+2)} \tag{19}$$

2. Trigonometric & Hyperbolic Functions

Function	Description
`sin`, `cos`, `tan`	Sine, cosine, tangent
`arcsin`, `arccos`, `arctan`	Inverse sine, cosine, tangent
`sinh`, `cosh`, `tanh`	Hyperbolic sine, cosine, tangent
`arcsinh`, `arccosh`, `arctanh`	Inverse hyperbolic sine, cosine, tangent
`sec`, `csc`, `cot`	Secant, cosecant, cotangent
`arcsec`, `arccsc`, `arccot`	Inverse secant, cosecant, cotangent
`sech`, `csch`, `coth`	Hyperbolic secant, cosecant, cotangent
`arcsech`, `arccsch`, `arccoth`	Inverse hyperbolic secant, cosecant, cotangent
`circexp(expr)`	Rewrites trig/hyperbolic functions in exponential form
`expcos(x)`, `expsin(x)`	`cos(x)` / `sin(x)` written directly in exponential form

```
sin(pi/6)             # exact value at a special angle

cos(pi/3)             # exact value at a special angle

tan(pi/4)             # exact value at a special angle

arcsin(1/2)           # angle whose sine is 1/2

float(arctanh(1/2))   # numeric value

circexp(sin(x))       # sine in exponential form

expsin(x)             # the same, written directly
```

$$\frac{1}{2} \tag{1}$$

$$\frac{1}{2} \tag{3}$$

$$1 \tag{5}$$

$$\frac{\pi}{6} \tag{7}$$

$$0.549306... \tag{9}$$

$$\frac{ie^{-ix}}{2} - \frac{ie^{ix}}{2} \tag{11}$$

$$\frac{ie^{-ix}}{2} - \frac{ie^{ix}}{2} \tag{13}$$

Secant, cosecant and cotangent are rewritten into the basic functions, so `sec(x)` is shown as `1/cos(x)`. Derivatives and numeric values therefore work right away. `arccot(0)` is not covered and stops with a division by zero.

```
sec(pi/3)              # 1/cos(pi/3)

sec(x)                 # shown as 1/cos(x)

derivative(sec(x),x)   # differentiates the rewritten form

csc(pi/6)              # 1/sin(pi/6)

cot(pi/4)              # cos(pi/4)/sin(pi/4)

arcsec(2)              # inverse secant

arccot(1)              # inverse cotangent
```

```
14
15 coth(x)                # shown as 1/tanh(x)
```

$$2 \tag{1}$$

$$\frac{1}{cos(x)} \tag{3}$$

$$\frac{sin(x)}{(cos(x)^2)} \tag{5}$$

$$2 \tag{7}$$

$$1 \tag{9}$$

$$\frac{\pi}{3} \tag{11}$$

$$\frac{\pi}{4} \tag{13}$$

$$\frac{1}{tanh(x)} \tag{15}$$

3. Calculus

Function	Description
`derivative(f, x [,n])`	`n`-th derivative of `f` with respect to `x` (default `n=1`)
`integral(f, x)`	Indefinite integral of `f` with respect to `x`
`defint(f, x, a, b)`	Definite integral of `f` over `x` from `a` to `b`
`limit(f, x, a [,dir])`	Limit of `f` as `x` approaches `a`, see section 3.1
`taylor(f, x, n, a)`	Taylor series of `f` around `x=a`, up to degree `n`
`sum(f, i, a, b)`	Sum of `f` over `i` from `a` to `b`, see section 3.2
`product(f, i, a, b)`	Product of `f` over `i` from `a` to `b`

```
derivative(sin(x)*cos(x),x)  # product rule

integral(x^2,x)              # antiderivative, without a constant

defint(x^2,x,0,1)            # area under x^2 between 0 and 1

limit(sin(x)/x,x,0)          # a classic limit

taylor(exp(x),x,3,0)         # Taylor polynomial of degree 3 around 0

sum(i,i,1,10)                # 1 + 2 + ... + 10

product(i,i,1,5)             # 1 * 2 * ... * 5
```

$$cos(x)^2 - sin(x)^2 \tag{1}$$

$$\frac{x^3}{3} \tag{3}$$

$$\frac{1}{3} \tag{5}$$

$$1 \tag{7}$$

$$\frac{x^3}{6} + \frac{x^2}{2} + x + 1 \tag{9}$$

$$55 \tag{11}$$

$$120 \tag{13}$$

3.1 Limits & Infinity

Function	Description
`inf`	Symbolic infinity, shown as ∞
`limit(f, x, a)`	Two-sided limit, the result may be `inf` or `-inf`
`limit(f, x, inf)`	Limit at infinity, likewise at `-inf`
`limit(f, x, a, 1)`	Limit from the right, `-1` for the limit from the left

Limits can be infinite, can be taken at infinity, and can be one-sided. Functions with jumps, such as `abs`, `sgn` and `floor`, are handled on each side separately.

```
limit(1/x^2,x,0)                # infinite from both sides

limit(1/x,x,0,1)                # from the right

limit(1/x,x,0,-1)               # from the left

limit(abs(x)/x,x,0,1)           # right side of the jump at 0

limit(tan(x),x,pi/2,-1)         # tan grows without bound

limit(log(x),x,0,1)             # log is only defined for x > 0

limit((2*x^2+1)/(x^2-3),x,inf)  # ratio of the leading coefficients

limit(x*sin(1/x),x,inf)         # sin(1/x) behaves like 1/x
```

$$\infty \tag{1}$$

$$\infty \tag{3}$$

$$-\infty \tag{5}$$

$$1 \tag{7}$$

$$\infty \tag{9}$$

$$-\infty \tag{11}$$

$$2 \tag{13}$$

$$1 \tag{15}$$

Arithmetic with `inf` absorbs finite numbers. Terms with an unknown symbol are left alone, because that symbol could itself be infinite.

```
inf+1     # finite numbers are absorbed

-3*inf    # a negative factor flips the sign

1/inf     # tends to zero

inf^2     # still infinite

inf>1000  # comparisons work too

inf+x     # unevaluated, x could be -inf
```

$$\infty \tag{1}$$

$$-\infty \tag{3}$$

$$0 \tag{5}$$

$$\infty \tag{7}$$

$$1 \tag{9}$$

$$\infty + x \tag{11}$$

Indeterminate forms, and limits that do not exist, stop with a message instead of returning a wrong value:

```
inf-inf              # indeterminate form

0*inf                # indeterminate form

limit(1/x,x,0)       # left and right limits differ

limit(abs(x)/x,x,0)  # jump at 0

limit(log(x),x,0)    # log is undefined for x < 0
```

```
Stop: indeterminate form: inf-inf
Stop: indeterminate form: 0*inf or inf/inf
Stop: limit: left and right limits differ — limit does not exist
Stop: limit: left and right limits differ — limit does not exist
Stop: limit: could not determine a real sign beside the point — try a
one-sided limit
```

3.2 Sums in Closed Form

With a symbolic bound, `sum` returns a formula when every term is polynomial or geometric in the index. The result is in expanded form, and `factor` turns it into the familiar textbook form. Any other sum stays unevaluated.

```
sum(k,k,1,n)              # closed form, expanded

factor(sum(k,k,1,n),n)    # the familiar n(n+1)/2

factor(sum(k^2,k,1,n),n)  # sum of the first n squares

sum(k,k,m,n)              # the lower bound can be symbolic too

sum(2^k,k,0,n)            # geometric sum

sum(x^k,k,0,n)            # geometric sum with a symbolic ratio

sum(2^k+k,k,1,n)          # geometric plus polynomial terms

sum(1/k,k,1,n)            # harmonic sum, no closed form
```

$$\frac{n}{2} + \frac{n^2}{2} \tag{1}$$

$$\frac{n(1+n)}{2} \tag{3}$$

$$\frac{n(1+n)(1+2n)}{6} \tag{5}$$

$$\frac{m}{2} + \frac{n}{2} - \frac{m^2}{2} + \frac{n^2}{2} \tag{7}$$

$$-1 + 2^{1+n} \tag{9}$$

$$\frac{(-1 + x^{1+n})}{(x-1)} \tag{11}$$

$$\frac{n}{2} + 2(-1 + 2^n) + \frac{n^2}{2} \tag{13}$$

$$\sum_{k=1}^{n} \frac{1}{k} \tag{15}$$

4. Linear Algebra & Tensors

Function	Description
`det(A)`	Determinant of a square matrix
`adj(A)`	Adjugate (classical adjoint) of a square matrix
`inv(A)`	Inverse of a square matrix
`transpose(A)`	Transpose of a matrix/tensor
`dot(A, B)` / `inner(A, B)`	Matrix (dot) product — `dot` is an alias for `inner`
`outer(a, b)`	Outer product of two vectors
`contract(A)`	Sum over repeated indices (trace, for a matrix)
`rank(A)`, `shape(A)`	Number of axes (2 for every matrix) / the size of each axis
`unit(n)`, `identity(n)`	`n` × `n` identity matrix
`trace(A)`	Sum of the diagonal
`charpoly(A, x)`	Characteristic polynomial `det(A - x I)`
`rref(A)`	Reduced row echelon form
`nullspace(A)`	Basis of the null space, one vector per row
`matrixrank(A)`	Rank of a matrix, the number of pivots
`hilbert(n)`	`n` × `n` Hilbert matrix
`eigenval(A)`, `eigenvec(A)`, `eigen(A)`	Eigenvalues / eigenvectors of a symmetric matrix

```
det([[1,2],[3,4]])        # 1*4 - 2*3

adj([[1,2],[3,4]])        # adjugate

inv([[1,2],[3,4]])        # adjugate divided by the determinant

```

```
transpose([[1,2],[3,4]])  # rows become columns

outer([1,2],[3,4])        # a matrix from two vectors

contract([[1,2],[3,4]])   # 1 + 4, the trace

hilbert(3)                # entries 1/(i+j-1)

eigenval([[2,1],[1,2]])   # symmetric matrix
```

$$-2 \tag{1}$$

$$\begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} \tag{3}$$

$$\begin{bmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{bmatrix} \tag{5}$$

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \tag{7}$$

$$\begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix} \tag{9}$$

$$5 \tag{11}$$

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix} \tag{13}$$

$$\begin{bmatrix} 3.000000... & 0.0 \\ 0.0 & 1.000000... \end{bmatrix} \tag{15}$$

Row reduction works on any matrix, square or not. Use `matrixrank` for the rank in the linear algebra sense, since `rank` counts the axes of a tensor. When the null space contains only the zero vector, `nullspace` returns that vector. The eigenvalues of a small matrix follow exactly from its characteristic polynomial.

```
identity(3)                          # 3x3 identity matrix

trace([[1,2],[3,4]])                 # 1 + 4

charpoly([[2,1],[1,2]],x)            # det(A - x I)

roots(charpoly([[2,1],[1,2]],x),x)   # its roots are the eigenvalues

rref([[1,2,3],[4,5,6],[7,8,9]])      # reduced row echelon form

nullspace([[1,2,3],[4,5,6],[7,8,9]]) # vectors v with A v = 0

matrixrank([[1,2],[2,4]])            # the second row is twice the f

rank([[1,2],[2,4]])                  # number of axes, not the matri
```

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1}$$

$$5 \tag{3}$$

$$x^2 - 4x + 3 \tag{5}$$

$$\begin{bmatrix} 1 & 3 \end{bmatrix} \tag{7}$$

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{bmatrix} \tag{9}$$

$$\begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \tag{11}$$

$$1 \tag{13}$$

$$2 \tag{15}$$

5. Polynomials & Number Theory

Function	Description
`factorpoly(p, x)`	Factors polynomial `p` over `x`
`roots(p, x)`, `nroots(p)`	Exact / numeric roots of polynomial `p`
`coeff(p, x, n)`	Coefficient of `x^n` in polynomial `p`
`deg(p, x)`, `leading(p, x)`	Degree of `p` in `x` / its leading coefficient
`gcd(a, b, ...)`, `lcm(a, b, ...)`	Greatest common divisor / least common multiple
`binomial(n, k)` / `choose(n, k)`	Binomial coefficient (the two names are equivalent)
`factorial(n)`	`n!`
`isprime(n)`, `prime(n)`, `divisors(n)`	Primality test / the `n`-th prime / all divisors of `n`

```
factorpoly(x^2-1,x)   # factors in x

roots(x^2-5*x+6,x)    # exact roots

coeff(x^2+3*x+5,x,1)  # coefficient of x^1

deg(x^3+2*x,x)        # highest power of x

gcd(12,18)            # greatest common divisor

lcm(4,6)              # least common multiple

binomial(5,2)         # 5 choose 2

factorial(6)          # same as 6!

isprime(17)           # 1 means prime

divisors(12)          # all divisors

prime(5)              # the 5th prime number
```

$$(x-1)(x+1) \tag{1}$$

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \tag{3}$$

$$3 \tag{5}$$

$$3 \tag{7}$$

$$6 \tag{9}$$

$$12 \tag{11}$$

$$10 \tag{13}$$

$$720 \tag{15}$$

$$1 \tag{17}$$

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 6 & 12 \end{bmatrix} \tag{19}$$

$$11 \tag{21}$$

6. Complex Numbers & Special Functions

Function	Description
`real(z)`, `imag(z)`	Real / imaginary part of a complex number
`conj(z)`	Complex conjugate
`arg(z)`, `abs(z)`	Argument (angle) / modulus of a complex number
`polar(z)`, `rect(z)`	Rewrites `z` in polar (`exp` form) / rectangular (`a+b*i`) form
`erf(x)`, `erfc(x)`	Error function and its complement
`Gamma(x)`	Gamma function
`besselj(x, n)`, `bessely(x, n)`	Bessel functions of the first / second kind
`dirac(x)`	Dirac delta function
`heaviside(x)`	Heaviside step function, `1/2` at `x=0`
`hermite(x, n)`, `laguerre(x, n)`, `legendre(x, n)`	Classical orthogonal polynomials of degree `n`

```
real(2+3*i)                 # real part

imag(2+3*i)                 # imaginary part

conj(2+3*i)                 # sign of the imaginary part flips

arg(2+3*i)                  # angle in the complex plane

abs(2+3*i)                  # distance from 0

polar(2+3*i)                # modulus times exp(i*angle)

float(erf(1))               # numeric value

Gamma(1/2)                  # exact value

besselj(0,1)                # J_1 at x = 0

dirac(0)                    # delta function at 0

```

```
heaviside(x)                # step function

derivative(heaviside(x),x)  # the step differentiates to a delta

hermite(x,3)                # Hermite polynomial of degree 3

legendre(x,2)               # Legendre polynomial of degree 2
```

$$2 \tag{1}$$

$$3 \tag{3}$$

$$2 - 3i \tag{5}$$

$$arctan(\frac{3}{2}) \tag{7}$$

$$\sqrt{13} \tag{9}$$

$$\sqrt{13}e^{iarctan(\frac{3}{2})} \tag{11}$$

$$0.842701... \tag{13}$$

$$\sqrt{\pi} \tag{15}$$

$$0 \tag{17}$$

$$1 \tag{19}$$

$$\frac{1}{2} + \frac{sgn(x)}{2} \tag{21}$$

$$dirac(x) \tag{23}$$

$$8x^3 - 12x \tag{25}$$

$$\frac{3}{2}x^2 - \frac{1}{2} \tag{27}$$

7. Solving Equations

Function	Description
`solve(p, x)`	Solves the polynomial equation `p=0` for `x`
`solve([p1, p2, ...], [x, y, ...])`	Solves a system of linear equations

A single equation must be a polynomial in one variable. A system must be linear, with as many equations as variables. Equations are written as expressions equal to zero or with `==`, because a single `=` would be an assignment.

```
solve(x^2-5*x+6,x)          # roots 2 and 3

solve(x^2-2,x)              # irrational roots

solve(x^3-6*x^2+11*x-6,x)   # cubic with three integer roots
```

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \tag{1}$$

$$\begin{bmatrix} -\sqrt{2} & \sqrt{2} \end{bmatrix} \tag{3}$$

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \tag{5}$$

The solution of a system lists the values in the order of the variables:

```
solve([x+y=3, x-y=1], [x, y])                    # equations written as
```

```
solve([x+y-3,x-y-1],[x,y])                    # equations written as
  expressions = 0

solve([x+y==3,x-y==1],[x,y])                  # the same system written
  ==

solve([x+y+z-6,x-y+z-2,2*x+y-z-1],[x,y,z])  # three equations, three
  unknowns
```

$$\begin{bmatrix} 2 & 1 \end{bmatrix} \tag{1}$$

$$\begin{bmatrix} 2 & 1 \end{bmatrix} \tag{3}$$

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \tag{5}$$

Systems without a unique solution, and nonlinear systems, stop with a message:

```
solve([x+y-1,2*x+2*y-2],[x,y])  # the second equation is twice the fir

solve([x*y-1,x-y],[x,y])        # x*y is not linear
```

```
Stop: solve: system has no unique solution
Stop: solve: system is not linear in the given variables
```

8. Logic, Comparison, Control Flow & Patterns

Function	Description
`and(a, b, ...)`, `or(a, b, ...)`, `not(a)`	Logical AND / OR / NOT, `1` or `0`
`testeq`, `testgt`, `testge`, `testlt`, `testle`	`1` / `0` comparisons: equal, greater/less (than or equal)
`test(cond, then, else)`	Returns `then` if `cond` is true, `else` otherwise
`for(body, i, a, b)`	Evaluates `body` repeatedly while `i` runs from `a` to `b`
`do(a, b, ...)`	Evaluates a sequence of expressions, returns the last one
`subst(a, x, expr)`	Substitutes `a` for `x` in `expr`
`pattern(from, to)`	Defines a rewrite rule, applied whenever `simplify()` runs
`clearpatterns`	Removes all user-defined patterns
`clear(x)`, `clearall`	Removes one binding / all bindings, patterns and settings

`for`'s first argument is the loop body, not the loop variable — the example below sums `1..5` into `s`. Wildcards in a `pattern()` template end with an underscore, e.g. `x_`.

```
clearall                    # reset all variables and settings

and(1,1,0)                  # 0, because one argument is false

or(0,0,1)                   # 1, because one argument is true

not(0)                      # negation

testeq(2+2,4)               # 1 means equal

testgt(5,3)                 # 1 means greater

s=0                         # start the sum at 0

for(s=s+i,i,1,5)            # add i = 1 ... 5 to s
```

```
for(s=s+i,i,1,5)                # add i = 1, ..., 5 to s
16
17  s                           # the result of the loop
18
19  subst(2,x,x^2+1)            # replace x by 2
20
21  pattern(sin(x_)^2+cos(x_)^2,1)  # x_ matches any expression
22
23  simplify(sin(a)^2+cos(a)^2)     # the rule applies here
24
25  clearpatterns               # remove the rule again
26
27  clearall                    # reset all variables and settings
```

```
0
1
1
1
1
15
5
sin(x_)^2+cos(x_)^2->1
1
```

9. Units & Physical Quantities

Function	Description
`quantity(value, unit)`	Creates a physical quantity with a unit
`convert(quantity, unit)`	Converts a quantity to another (compatible) unit
`dimensionof(quantity)`	SI base-unit dimension vector of a quantity
`units(0|1)`	Toggles whether bare symbols like `m`, `kg`, `s` are treated as units

`quantity()` works regardless of the `units()` setting; arithmetic on quantities enforces dimensional consistency (adding metres and kilograms is an error).

```
1  clearall                    # reset all variables and settings
2
3  quantity(5,m)+quantity(200,cm)  # the units are converted before addi
4
```

```
convert(quantity(5,m),cm)       # metres to centimetres

quantity(5,m)*quantity(2,s)     # units are multiplied too

dimensionof(quantity(5,m))      # exponents of the SI base units

units(1)                        # treat m, cm, s, ... as units

5*m+200*cm                      # works without quantity() now

units(0)                        # back to plain symbols

clearall                        # reset all variables and settings
```

```
7*m
500
10*m*s
[1,0,0,0,0,0,0]
1
7*m
0
```

10. Output & Utility Functions

Function	Description
`numerator(x)`, `denominator(x)`	Numerator / denominator of a rational expression
`round(x)`, `ceiling(x)`, `floor(x)`	Rounds to the nearest / next higher / next lower integer
`mod(a, b)`	Remainder of `a` divided by `b`
`sgn(x)`	Sign of `x` (`-1`, `0` or `1`)
`print(...)`, `printhuman(expr)`, `printcomputer(expr)`	Prints in default / traditional math / fully explicit notation
`print2dascii(expr)`	Renders an expression as 2D ASCII art (fractions, exponents)
`printlatex(expr)`	Renders an expression as a LaTeX string — this is what powers `@Algebrite.pretty`

```
numerator(3/7)    # top of the fraction

denominator(3/7)  # bottom of the fraction

round(3.6)        # nearest integer

ceiling(3.2)      # next integer up

floor(3.8)        # next integer down

mod(10,3)         # remainder of 10/3

sgn(-5)           # sign of a negative number
```

```
3
7
4.0
4.0
3.0
1
-1
```

`printhuman` and `printcomputer` show the same expression in two different notations — one for reading, one for re-parsing:

```
printhuman(2*x^2+3*x)  # for reading
```

```
3 x + 2 x^2
```

```
printcomputer(2*x^2+3*x)  # for re-parsing
```

```
3*x+2*x^2
```

```
print2dascii(2*x^2+3*x)  # exponents drawn in 2D
```

```
        2
3 x + 2 x
```

```
printlatex(sin(x)^2+cos(x)^2)  # LaTeX source
```

```
cos(x)^2+sin(x)^2
```

11. Plotting with `draw`

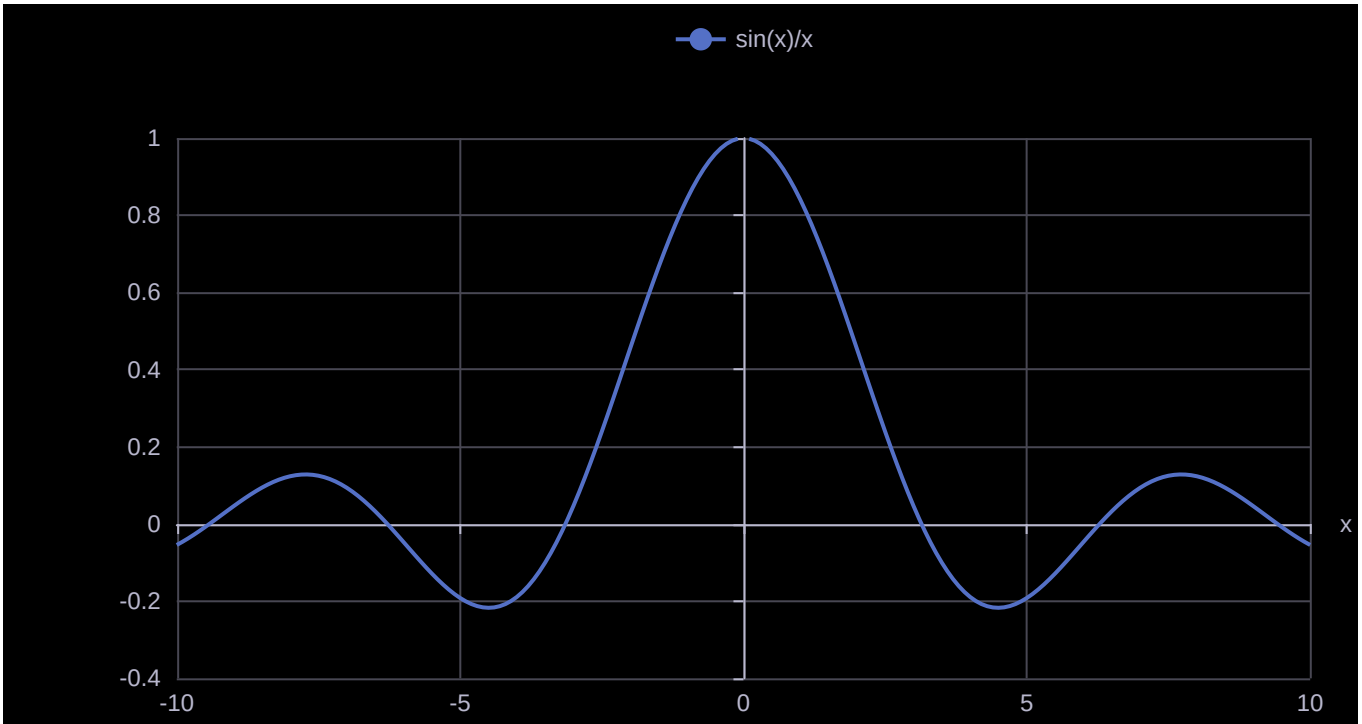
`draw` plots expressions as an interactive chart into the console of the code block. It returns nothing, so it can be mixed freely with other expressions.

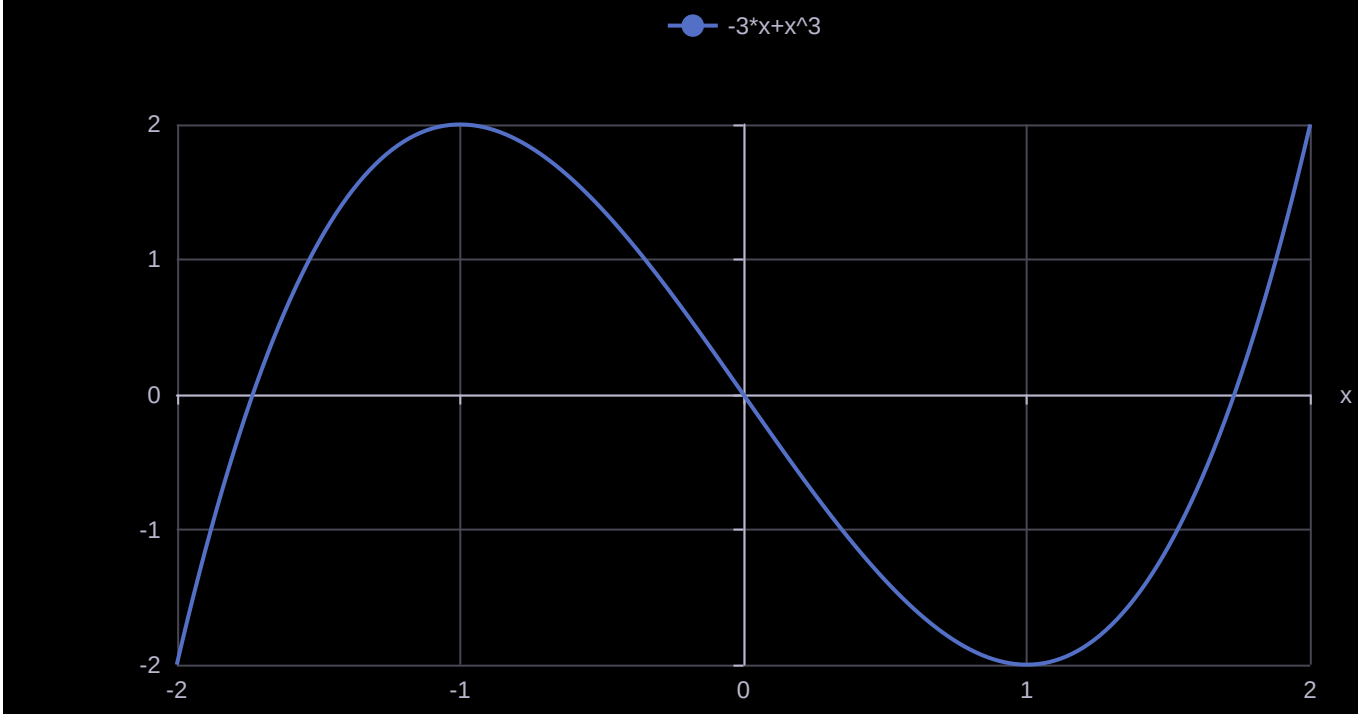
Call	Description
`draw(f)`	Plots `f` over `x` from `-10` to `10`
`draw(f, t)`	Plots `f` over the variable `t`
`draw(f, x, a, b)`	Plots `f` over `x` from `a` to `b`, the limits may be symbolic
`draw([f, g, ...], x, a, b)`	Several curves in one chart, with a legend
`draw(f, x, a, b, "options")`	Styles the curve, see the tables below
`draw([f, g], x, a, b, ["options f", "options g"])`	One option string per curve

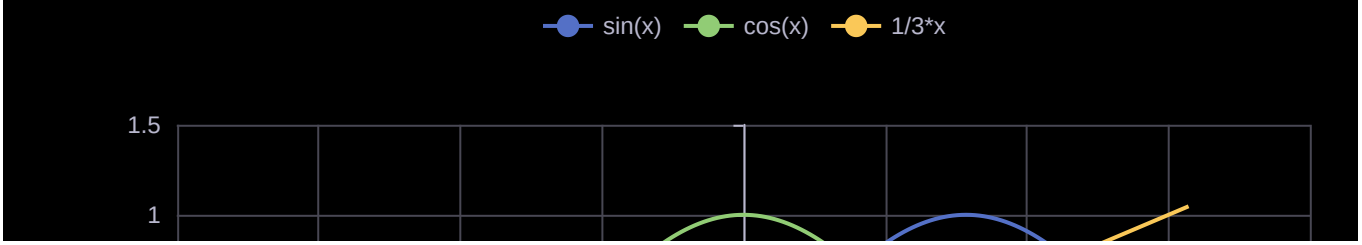
```
draw(sin(x)/x, x, -10, 10)                    # a single curve

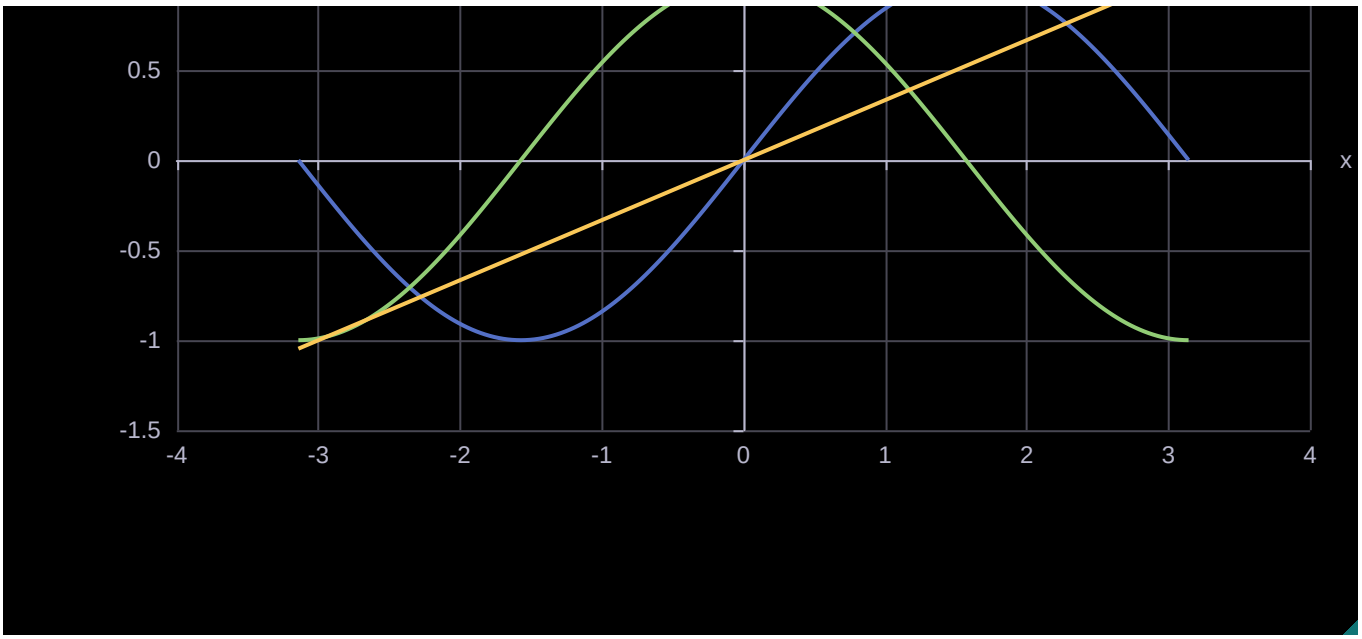
f(x) = x^3 - 3*x                              # user-defined functions wo
draw(f(x), x, -2, 2)

draw([sin(x), cos(x), x/3], x, -pi, pi)       # several curves with a leg
```


sin(x)/x
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-10
-5
0
5
10
x


-3*x+x^3
2
1
0
-1
-2
-2
-1
0
1
2
x


sin(x)
cos(x)
1/3*x
1.5
1

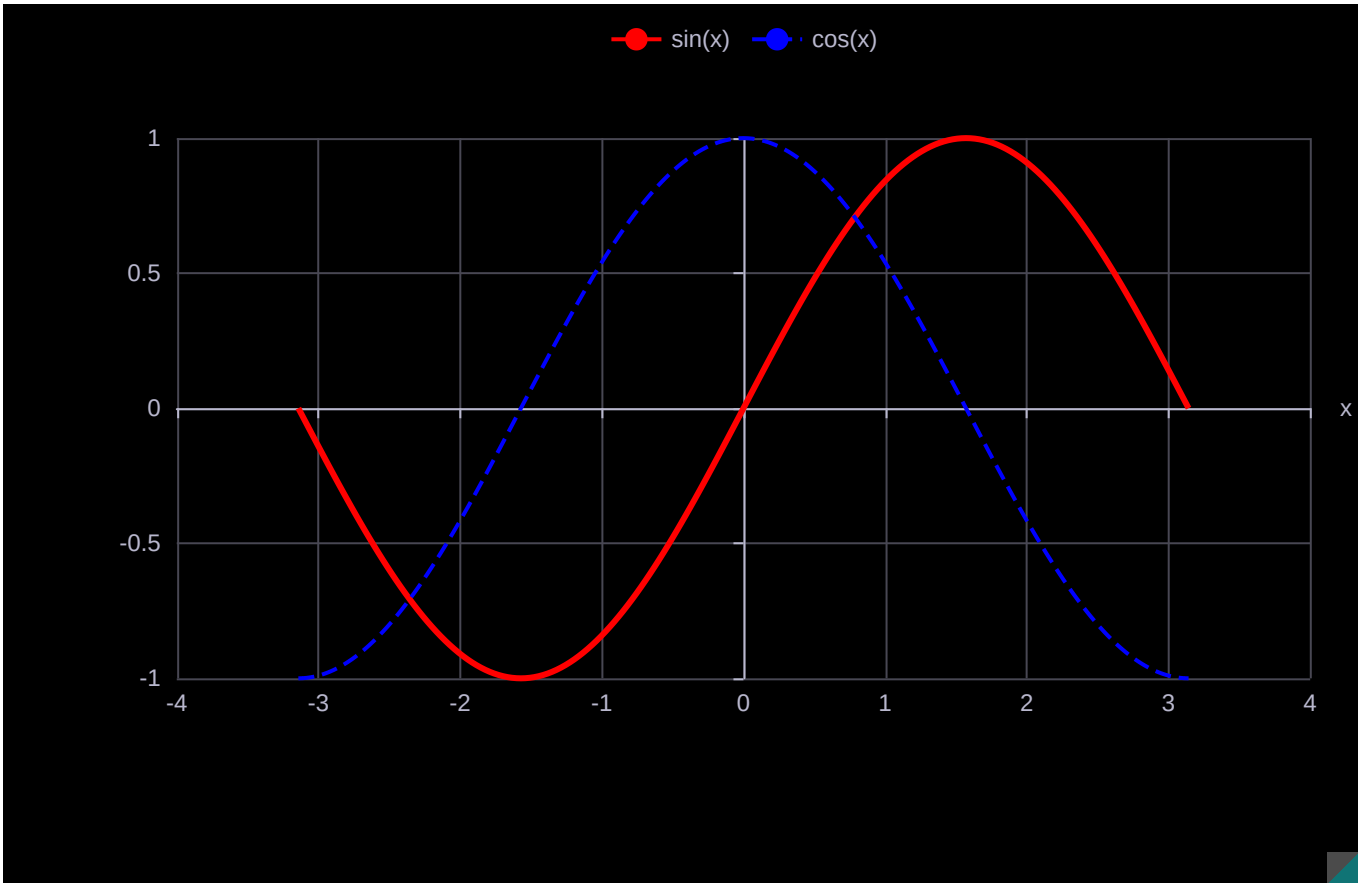


11.1 Line Style

Options are words in a string, separated by spaces. A word must not contain a space itself: write `width=3`, not `width = 3`.

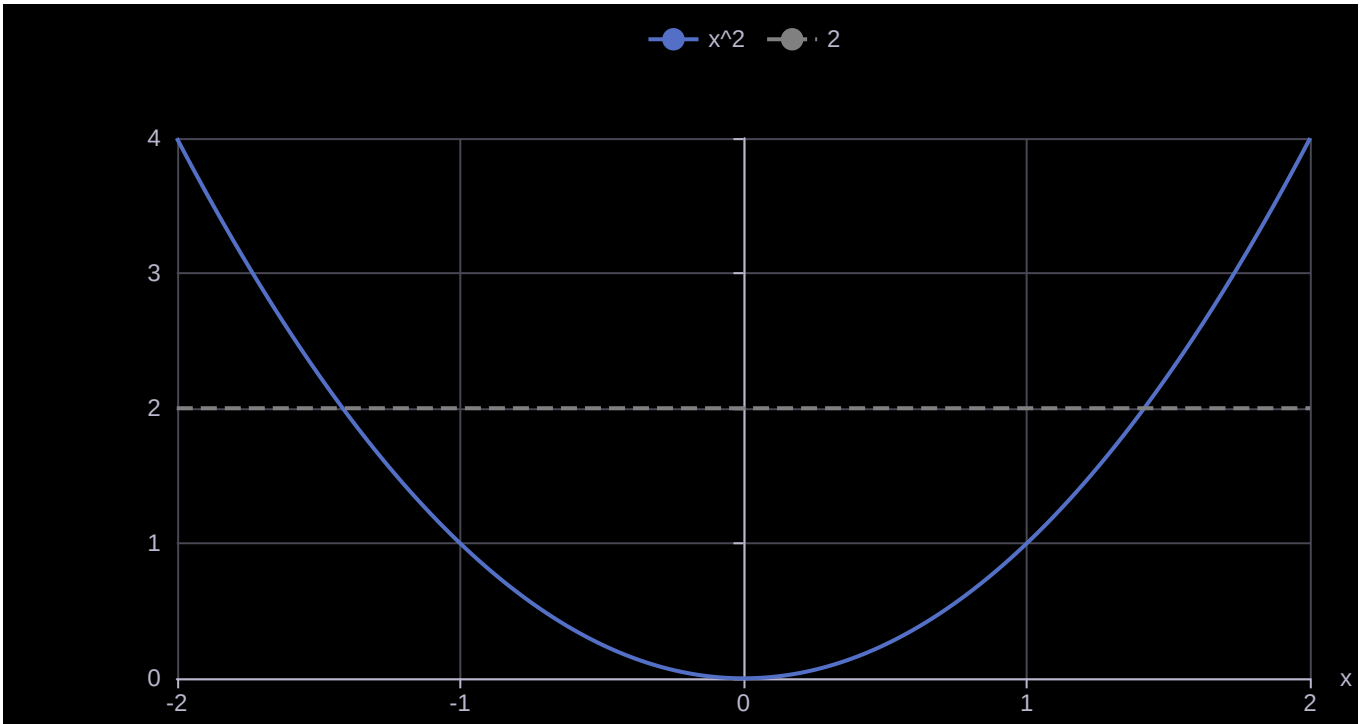
Option	Description
`red`, `blue`, `#ff8800`, …	Line color, any CSS color name or hex code
`solid`, `dashed`, `dotted`	Line type (default `solid`)
`width=n`	Line width in pixels (default `2`)

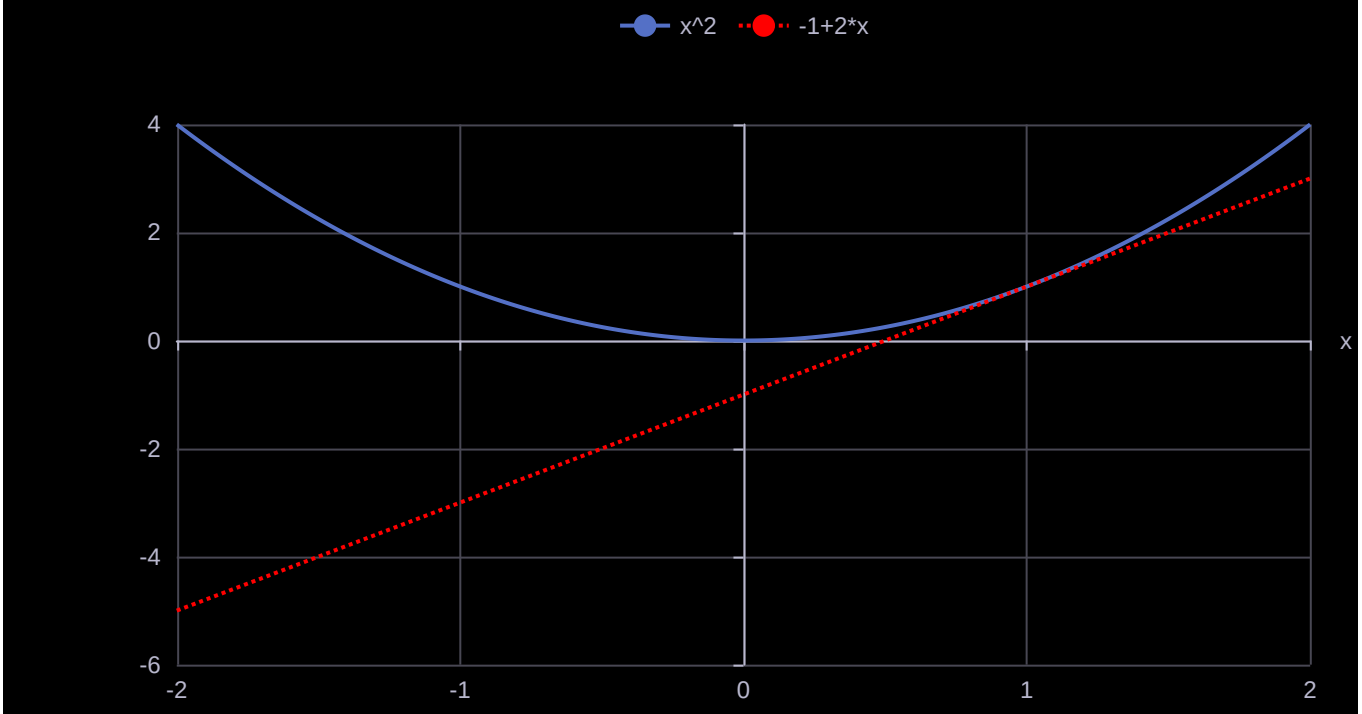
```
1 draw([sin(x), cos(x)], x, -pi, pi, ["red width=3", "blue dashed"])
```

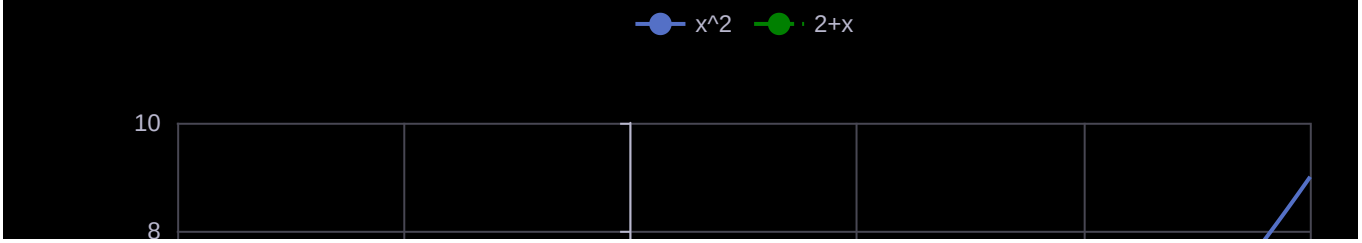


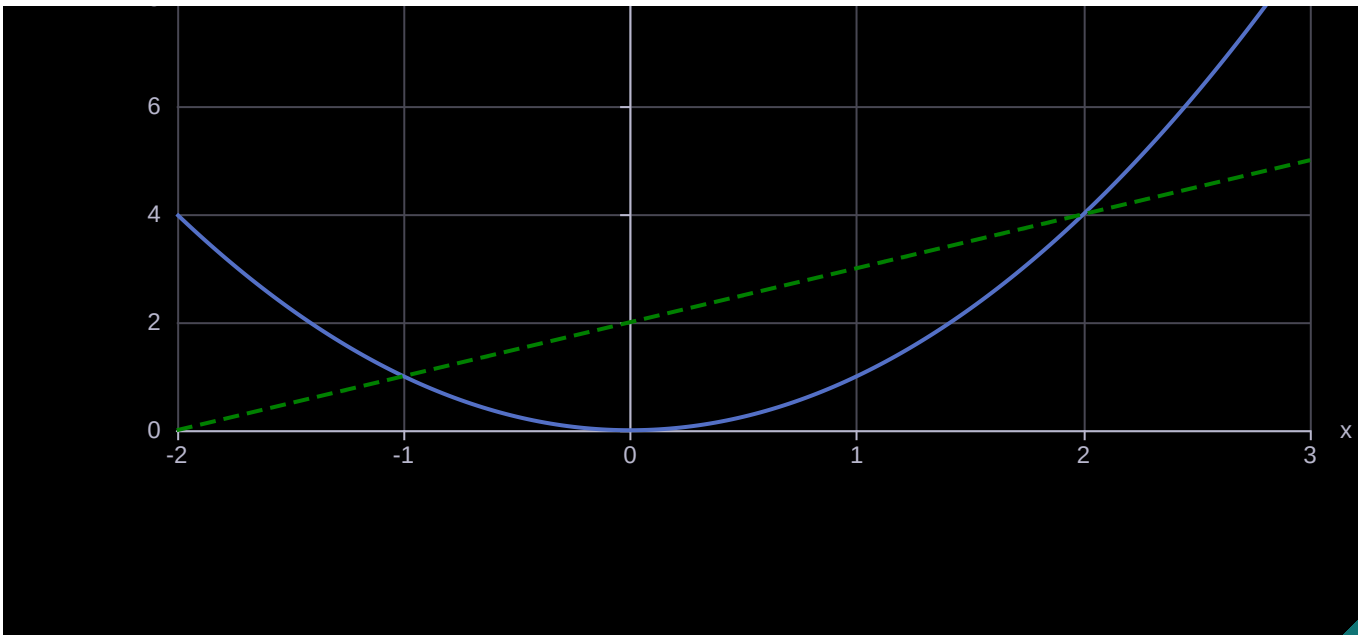
Straight lines that are functions of `x`, such as a constant, a tangent or a secant, are simply drawn as another curve:

```
f(x) = x^2
draw([f(x), 2], x, -2, 2, ["", "gray dashed"])              # horizont
  line y=2
draw([f(x), 2*x-1], x, -2, 2, ["", "red dotted"])           # tangent
draw([f(x), x+2], x, -2, 3, ["", "green dashed"])           # secant t
  x=-1 and x=2
```


x^2
2
4
3
2
1
0
-2
-1
0
1
2
x


x^2
-1+2*x
4
2
0
-2
-4
-6
-2
-1
0
1
2
x


x^2
2+x
10
8



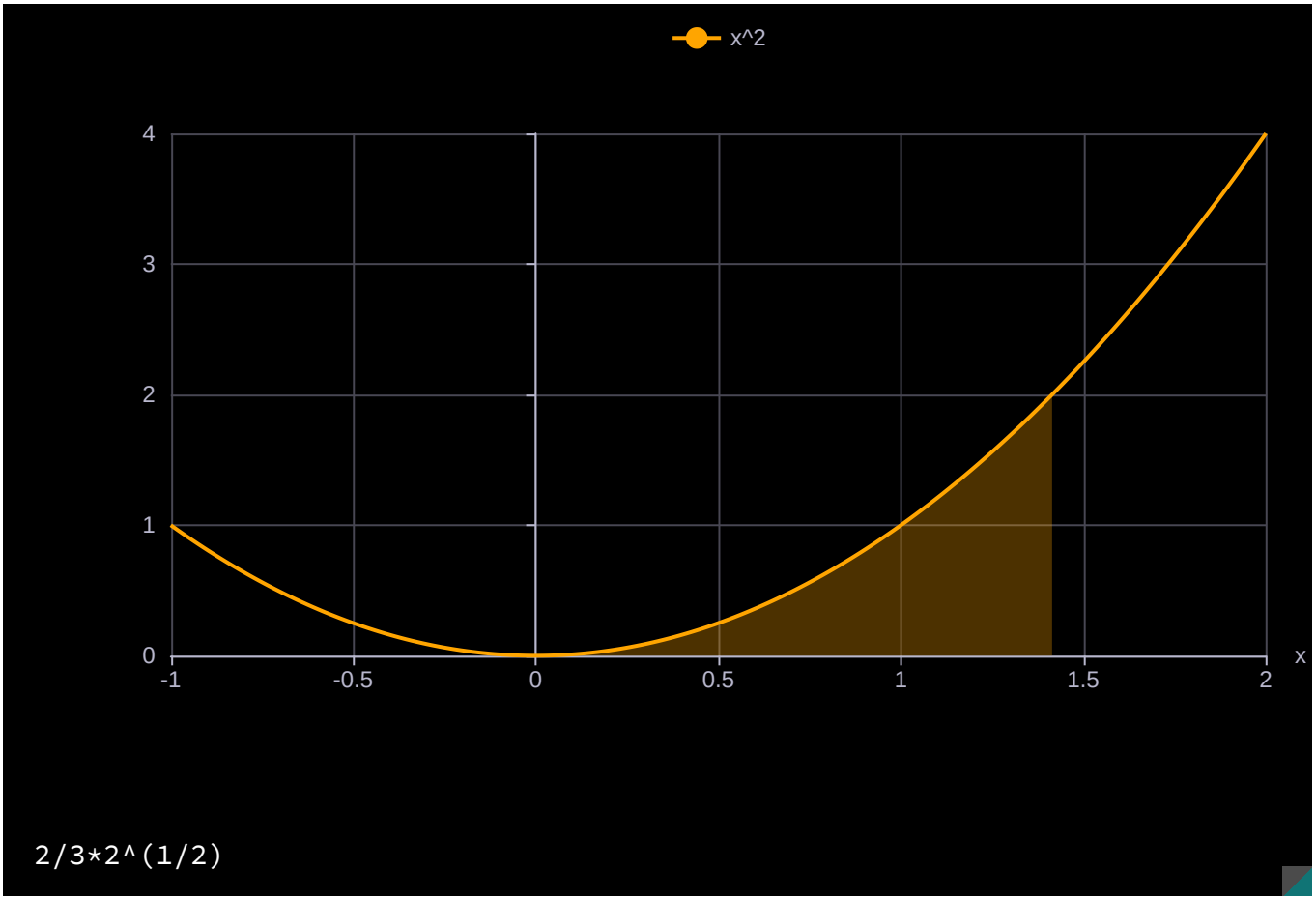
11.2 Filled Areas

Option	Description
`fill`	Fills the area between the curve and the x-axis
`fill=a..b`	Fills only between `x=a` and `x=b`, the limits may be symbolic

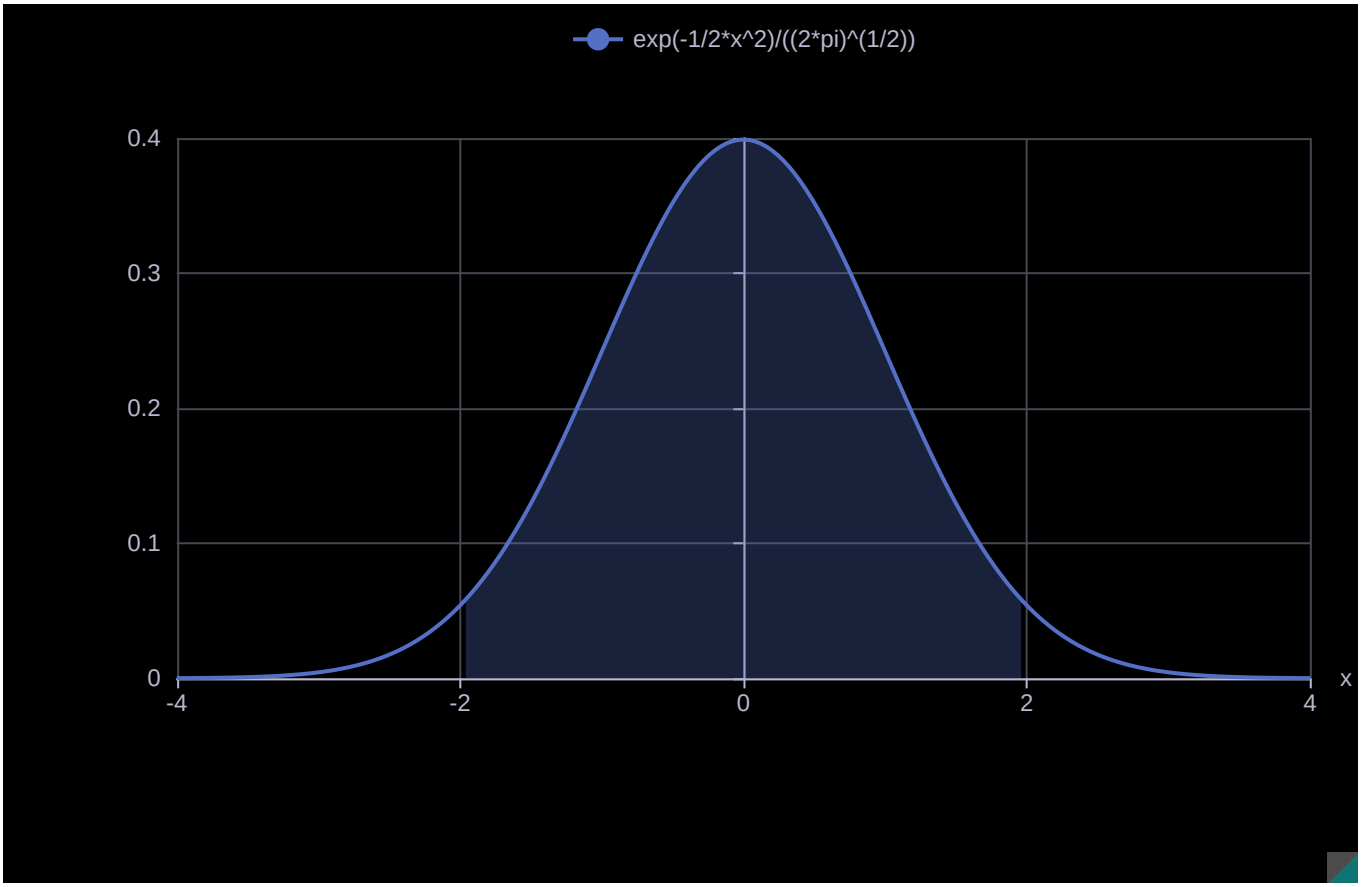
A filled interval visualizes a definite integral or a probability:

```
draw(x^2, x, -1, 2, "orange fill=0..sqrt(2)")   # area of the integral

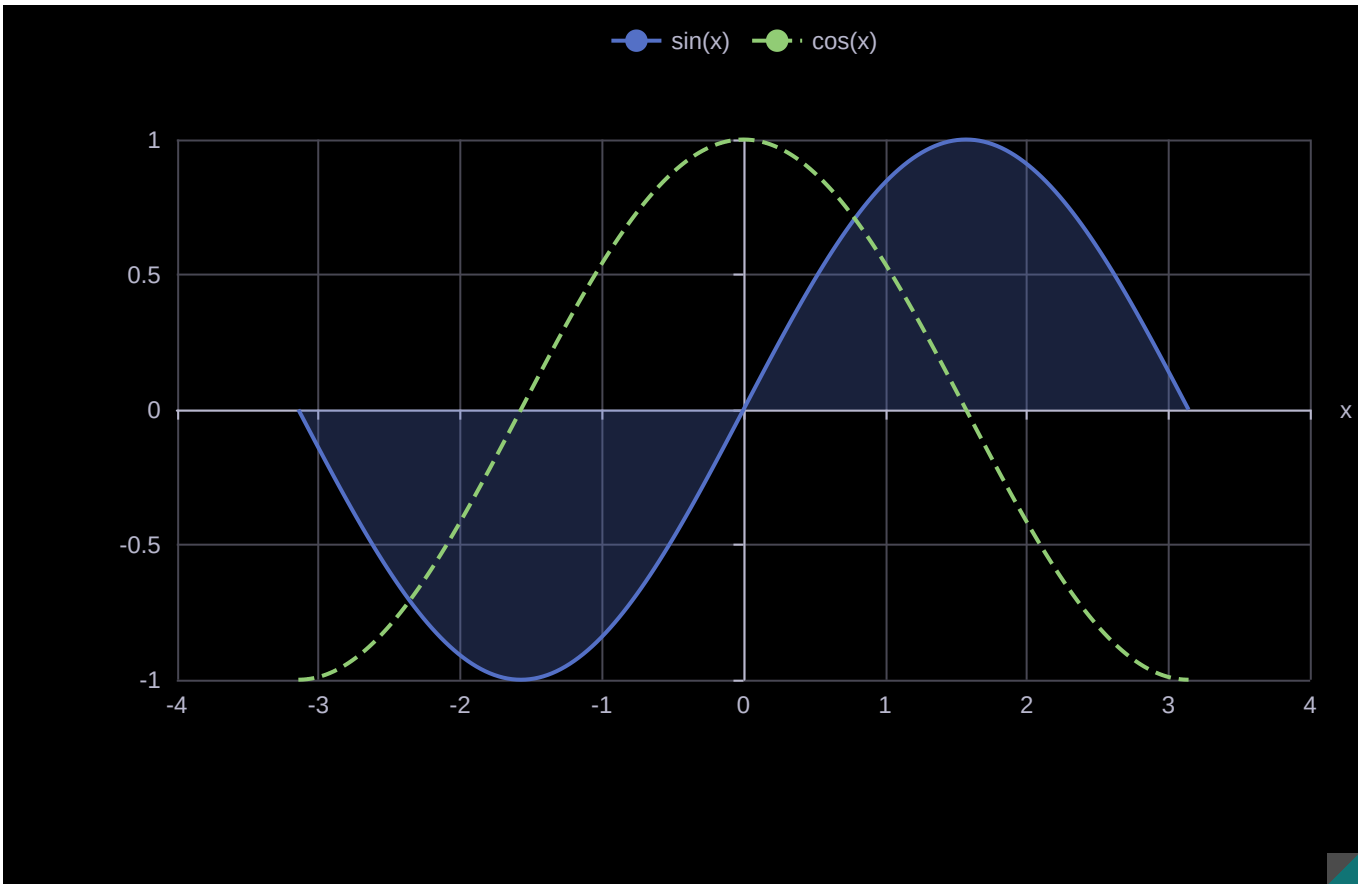
defint(x^2, x, 0, sqrt(2))
```




```
1  # standard normal distribution, P(-1.96 < X < 1.96) is about 95 %
2  draw(exp(-x^2/2)/sqrt(2*pi), x, -4, 4, "fill=-1.96..1.96")
```




```
1  draw([sin(x), cos(x)], x, -pi, pi, ["fill", "dashed"])
```

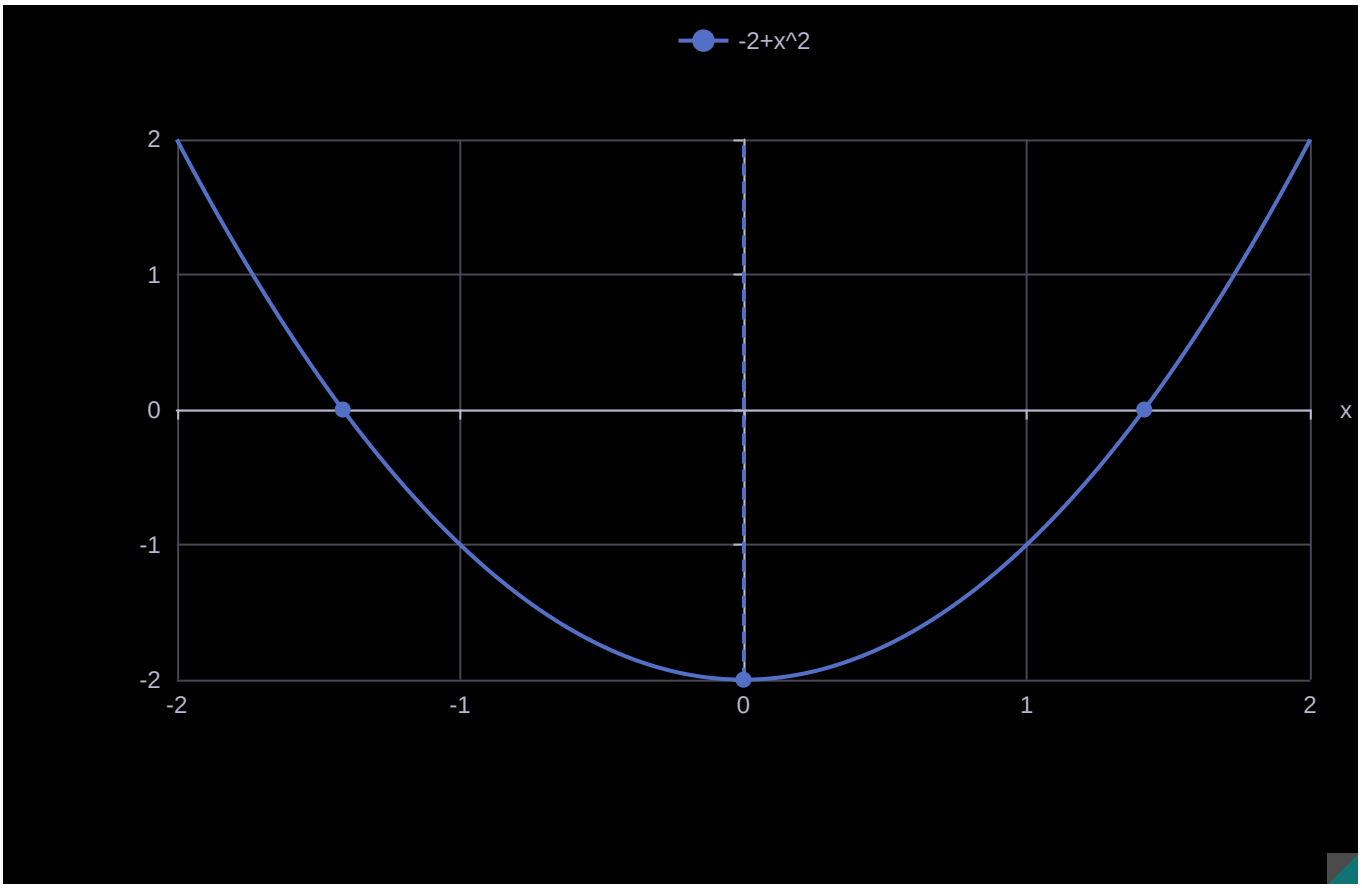


11.3 Points and Lines

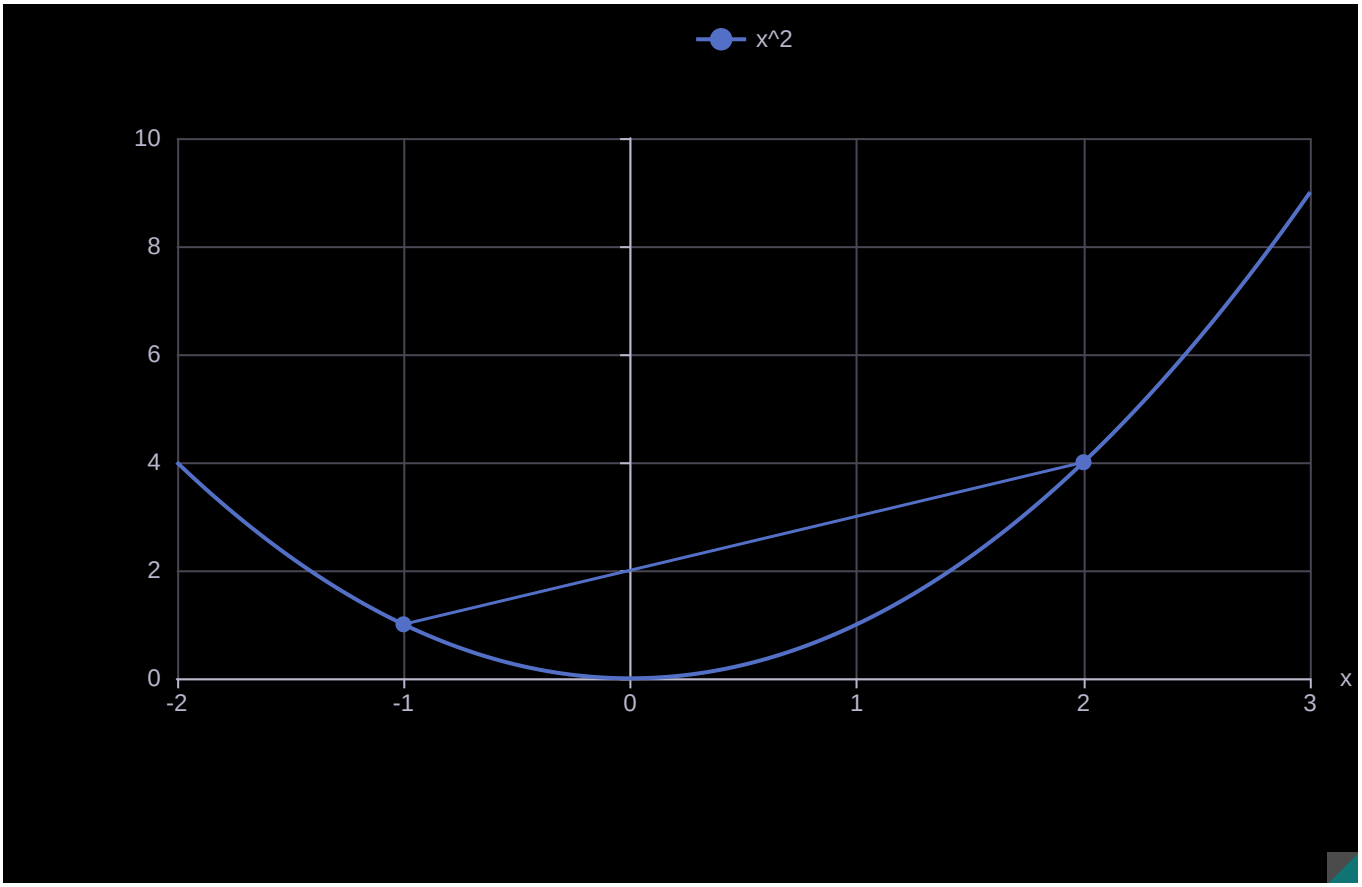
Option	Description
`mark=a`	Point on the curve at `(a, f(a))`
`point=x,y`	Point at `(x, y)`
`vline=a`	Vertical dashed line at `x=a`
`line=x1,y1,x2,y2`	Line segment from `(x1, y1)` to `(x2, y2)`

These options can be repeated and take the color of their curve. The values may be symbolic, but they must not contain a comma: `point=1,f(1)` works, `point=1,max(2,3)` does not.

```
f(x) = x^2 - 2
draw(f(x), x, -2, 2, "mark=-sqrt(2) mark=sqrt(2) vline=0 point=0,-2")
```




```
# secant slope between two points of x^2
draw(x^2, x, -2, 3, "mark=-1 mark=2 line=-1,1,2,4")
```



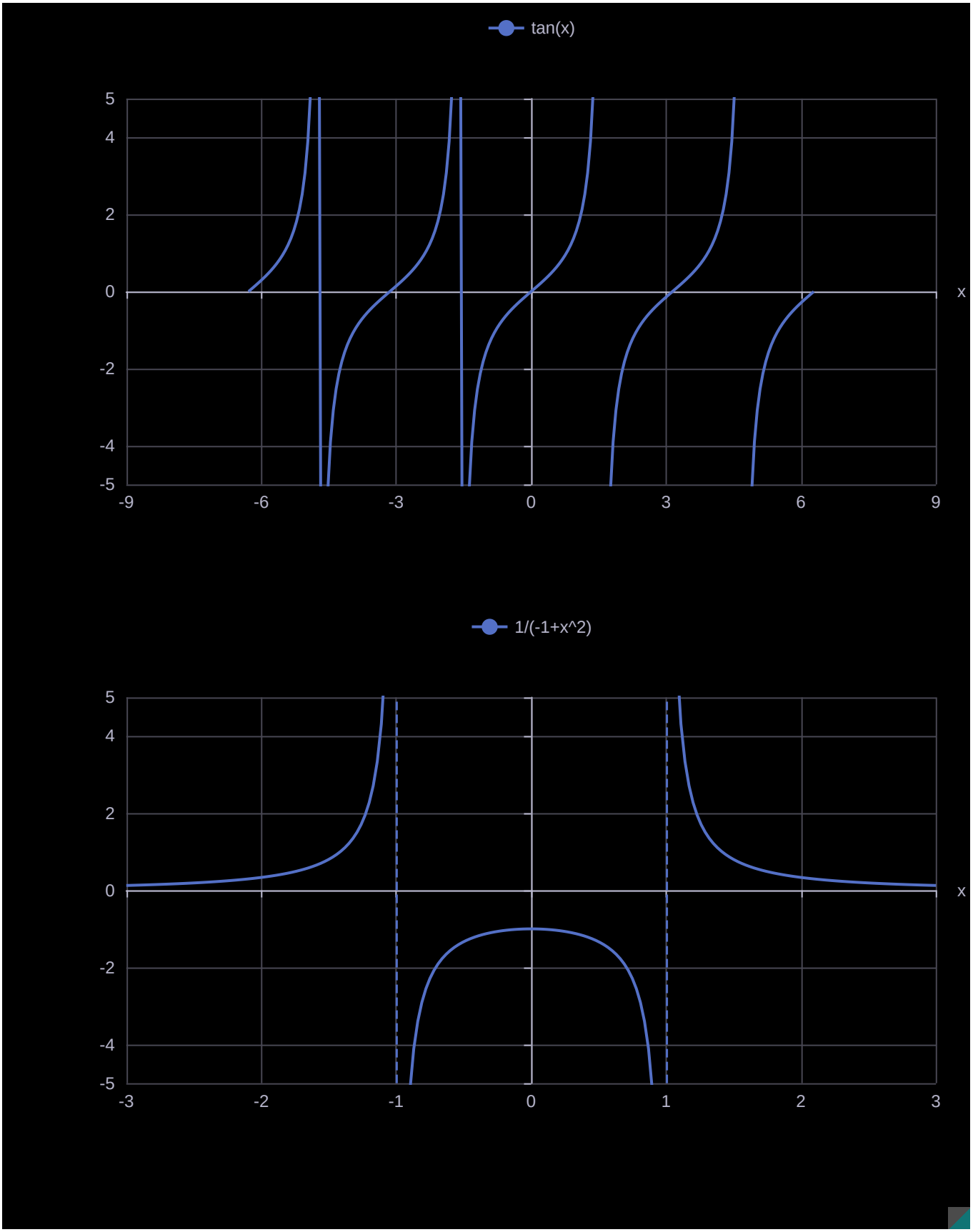
11.4 Axis Range and Poles

Option	Description
`ymin=a`, `ymax=b`	Limits the visible y-range, the limits may be symbolic

At poles, the curve is interrupted instead of connected by a vertical line. Without `ymin`/`ymax`, the huge values next to a pole squeeze the rest of the curve, so it is best to set the y-range there:

```
draw(tan(x), x, -2*pi, 2*pi, "ymin=-5 ymax=5")

draw(1/(x^2-1), x, -3, 3, "ymin=-5 ymax=5 vline=-1 vline=1")
```



Function Index (A–Z)

Every function available in Algebrite expressions, alphabetically. Functions defined as formulas over other functions, such as `sec` or `log10`, work in expressions but are not methods of the JavaScript `Algebrite` object. A few are reserved for future use and currently do nothing (`dsolve`, `guess`); `decomp` is experimental and may misbehave.

Function	Description
`abs(x)`	Absolute value / complex modulus
`add(a, b, ...)`	Sum of two or more terms
`adj(A)`	Adjugate of a square matrix
`and(a, b, ...)`	Logical AND
`apart(f, x)`	Partial fraction decomposition
`approxratio(x)`	Approximates a float as a rational number
`arccos(x)`	Inverse cosine
`arccosh(x)`	Inverse hyperbolic cosine
`arccot(x)`	Inverse cotangent
`arccoth(x)`	Inverse hyperbolic cotangent
`arccsc(x)`	Inverse cosecant
`arccsch(x)`	Inverse hyperbolic cosecant
`arcsec(x)`	Inverse secant
`arcsech(x)`	Inverse hyperbolic secant
`arcsin(x)`	Inverse sine
`arcsinh(x)`	Inverse hyperbolic sine
`arctan(x)`	Inverse tangent
`arctanh(x)`	Inverse hyperbolic tangent
`arg(z)`	Argument (angle) of a complex number
`atomize(expr)`	Top-level arguments of an expression as a vector
`besselj(x, n)`	Bessel function of the first kind

`bessely(x, n)`	Bessel function of the second kind
`binding(x)`	Current value bound to a symbol
`binomial(n, k)`	Binomial coefficient
`cbrt(x)`	Cube root
`ceiling(x)`	Rounds up to the nearest integer
`charpoly(A, x)`	Characteristic polynomial of a matrix
`check(expr)`	Evaluates a relational expression to `1`/`0`
`choose(n, k)`	Alias for `binomial`
`circexp(expr)`	Rewrites trig/hyperbolic functions in exponential form
`clear(x)`	Removes the binding of a symbol
`clearall`	Clears all bindings, patterns and settings
`clearpatterns`	Removes all user-defined patterns
`clock(x)`	Converts a number to `exp(i*pi*n)` (“clock”) form
`coeff(p, x, n)`	Coefficient of `x^n` in polynomial `p`
`cofactor(A, i, j)`	Cofactor of `A` at row `i`, column `j`
`condense(expr)`	Factors common terms out of a sum
`conj(z)`	Complex conjugate
`contract(A)`	Sum over repeated tensor indices
`convert(q, unit)`	Converts a physical quantity to another unit
`cos(x)`	Cosine
`cosh(x)`	Hyperbolic cosine

`cot(x)`	Cotangent
`coth(x)`	Hyperbolic cotangent
`csc(x)`	Cosecant
`csch(x)`	Hyperbolic cosecant
`decomp(p, x)`	Polynomial decomposition (experimental)
`defint(f, x, a, b)`	Definite integral
`deg(p, x)`	Degree of polynomial `p` in `x`
`denominator(x)`	Denominator of a rational expression
`det(A)`	Determinant of a square matrix
`derivative(f, x [,n])`	`n`-th derivative of `f`
`dim(A, n)`	Size of the `n`-th axis of a tensor
`dimensionof(q)`	SI dimension vector of a quantity
`dirac(x)`	Dirac delta function
`divisors(n)`	All divisors of an integer
`do(a, b, ...)`	Evaluates a sequence, returns the last result
`dot(A, B)`	Alias for `inner`
`draw(f, x, a, b, options)`	Plots expressions as a chart, see section 11
`dsolve`	Reserved, not implemented
`eigen(A)`	Eigenvalues and eigenvectors of a symmetric matrix
`eigenval(A)`	Eigenvalues of a symmetric matrix
`eigenvec(A)`	Eigenvectors of a symmetric matrix
`equals(a, b)`	Functional form of assignment, `a=b`

`erf(x)`	Error function
`erfc(x)`	Complementary error function
`eval(expr [,x, a])`	Evaluates an expression, optionally substituting `x=a` first
`exp(x)`	Exponential function `e^x`
`expand(expr)`	Multiplies out products and powers
`expcos(x)`	`cos(x)` in exponential form
`expsin(x)`	`sin(x)` in exponential form
`factor(expr)`	Factors a polynomial or integer
`factorial(n)`	`n!`
`factorpoly(p, x)`	Factors polynomial `p` over `x`
`filter(expr, x)`	Removes terms containing `x`, or zeroes matching tensor entries
`float(x)`	Numeric (floating point) evaluation
`floor(x)`	Rounds down to the nearest integer
`for(body, i, a, b)`	Repeats `body` while `i` runs from `a` to `b`
`Gamma(x)`	Gamma function
`gcd(a, b, ...)`	Greatest common divisor
`heaviside(x)`	Heaviside step function
`hermite(x, n)`	Physicists' Hermite polynomial
`hilbert(n)`	`n` × `n` Hilbert matrix
`identity(n)`	`n` × `n` identity matrix, same as `unit(n)`
`imag(z)`	Imaginary part of a complex number

`component(A, i, ...)`	Accesses a component of a tensor by index
`inf`	Symbolic infinity
`inner(A, B)`	Matrix / dot product
`integral(f, x)`	Indefinite integral
`inv(A)`	Inverse of a square matrix
`invg(x)`	Advanced special function, rarely needed directly
`isinteger(x)`	`1` if `x` is an integer, else `0`
`isprime(n)`	`1` if `n` is prime, else `0`
`laguerre(x, n)`	Laguerre polynomial
`lcm(a, b, ...)`	Least common multiple
`leading(p, x)`	Leading coefficient of polynomial `p`
`legendre(x, n)`	Legendre polynomial
`limit(f, x, a [,dir])`	Limit, also at `inf` and one-sided
`log(x [,b])`	Natural logarithm, or logarithm to base `b`
`log10(x)`	Logarithm to base 10
`log2(x)`	Logarithm to base 2
`matrixrank(A)`	Rank of a matrix
`max(a, b, ...)`	Largest argument
`min(a, b, ...)`	Smallest argument
`mod(a, b)`	Remainder of `a` divided by `b`
`multiply(a, b, ...)`	Product of two or more factors
`not(a)`	Logical negation

`nroots(p)`	All numeric (real and complex) roots of `p`
`nullspace(A)`	Basis of the null space of a matrix
`number(x)`	`1` if `x` is a number, else `0`
`numerator(x)`	Numerator of a rational expression
`operator(f)`	Marks a symbol as an operator (advanced)
`or(a, b, ...)`	Logical OR
`outer(a, b)`	Outer product of two vectors
`partfrac(f, x)`	Alias for `apart`
`pattern(from, to)`	Defines a rewrite rule used by `simplify()`
`patternsinfo()`	Lists all currently defined patterns
`polar(z)`	Rewrites a complex number in polar form
`power(a, b)`	`a^b`
`prime(n)`	The `n`-th prime number
`print(...)`	Prints one or more expressions
`print2dascii(expr)`	Renders an expression as 2D ASCII art
`printcomputer(expr)`	Prints in fully explicit (“computer”) notation
`printlatex(expr)`	Renders an expression as a LaTeX string
`printlist(list)`	Prints a list/tensor entry by entry
`printhuman(expr)`	Prints in traditional human-readable notation
`product(f, i, a, b)`	Product of `f` over `i` from `a` to `b`
`quantity(value, unit)`	Creates a physical quantity with a unit
`quote(expr)`	Returns `expr` unevaluated

`quotient(a, b)`	Quotient of `a` divided by `b`
`rank(A)`	Number of axes of a tensor, see `matrixrank` for matrices
`rationalize(expr)`	Combines terms over a common denominator
`real(z)`	Real part of a complex number
`rect(z)`	Rewrites a complex number in rectangular form
`root(x, n)`	`n`-th root
`roots(p, x)`	Rational/exact roots of polynomial `p`
`round(x)`	Rounds to the nearest integer
`rref(A)`	Reduced row echelon form
`sec(x)`	Secant
`sech(x)`	Hyperbolic secant
`shape(A)`	Dimensions of a tensor, as a list
`sgn(x)`	Sign of `x`
`silentpattern(from, to)`	Like `pattern()`, without a confirmation printout
`simplify(expr)`	General purpose simplification
`sin(x)`	Sine
`sinh(x)`	Hyperbolic sine
`solve(p, x)`	Solves a polynomial equation, or a linear system given as lists
`sqrt(x)`	Square root
`stop(msg)`	Aborts evaluation with an error message
`subst(a, x, expr)`	Substitutes `a` for `x` in `expr`

`sum(f, i, a, b)`	Sum of `f` over `i` from `a` to `b`, closed form for symbolic bounds
`symbolsinfo()`	Lists all currently bound symbols
`tan(x)`	Tangent
`tanh(x)`	Hyperbolic tangent
`taylor(f, x, n, a)`	Taylor series of `f` around `x=a`, degree `n`
`test(cond, then, else)`	Returns `then` or `else` depending on `cond`
`testeq(a, b)`	`1` if `a=b`, else `0`
`testge(a, b)`	`1` if `a>=b`, else `0`
`testgt(a, b)`	`1` if `a>b`, else `0`
`testle(a, b)`	`1` if `a<=b`, else `0`
`testlt(a, b)`	`1` if `a<b`, else `0`
`trace(A)`	Sum of the diagonal of a matrix
`transpose(A)`	Transpose of a matrix/tensor
`unit(n)`	`n` × `n` identity matrix
`units(0|1)`	Toggles bare unit symbols on/off
`zero(n)`	Zero vector of length `n`

Implementation

Compared to other macros, using Algebrite is actually quite simple. The `script` line loads the bundled Algebrite library together with the LaTeX converter, all macros are defined below.

```
script:   dist/index.js

attribute: [Algebrite](http://algebrite.org/)
           by [Davide Della Casa](http://davidedc.com/)
           is licensed under [MIT](https://opensource.org/licenses/MIT)
```

```
@onload
window.inputClean = function(input) {
  const commas = [",", ",", "⌧", "⌧", "⌧"];
  for(let i=0; i<commas.length; i++) {
    input = input.replace(new RegExp("\\" + commas[i], "g"), ".");
    input = input.replace/(\d+(?:\.\d+)?)\s*%/g, (_, num) => (parseFloat(r
      100).toString());
  }

  input = input.replace(/\\/g, "/");

  return input;
}

window.normalizeInputToArray = function(raw) {
  // Always start from a string
  let input = String(raw ?? "").trim();

  // If it might be JSON text, first make it JSON-safe (turn \frac -> \\fra
    .)
  const looksLikeJson = /^[\[\{"]/.test(input);
  if (looksLikeJson) input = input.replace(/\\/g, "\\\\");

  // Try JSON parsing only when it looks like JSON
  if (looksLikeJson) {
    try {
      const parsed = JSON.parse(input);

      // JSON string -> wrap into array
      if (typeof parsed === "string") return [parsed.trim()];

      // JSON array -> normalize items to strings
      if (Array.isArray(parsed)) return parsed.map(x => String(x).trim());

      // Anything else (object/number/etc.) -> stringify-ish fallback
      return [String(parsed).trim()];
    } catch (e) {
      // fall through to plain-string handling
    }
  }

  // Plain string input (e.g. \frac{12}{1}) -> just wrap
  // If you want to also "fix" \frac into \\frac for storage/transport, app
    escape here too:
  // input = window.escapeNonJsonBackslashes(input);
  return [input.trim()];
}

// Runs fn with units(1|0) and restores the previous setting afterwards,
// errors (e.g. incompatible units) count as a wrong answer.
```

```
window.algebriteUnits = function(on, fn) {
  const prev = window.Algebrite.run("units()");
  window.Algebrite.run(`units(${on ? 1 : 0})`);
  try {
    return fn();
  } catch (e) {
    console.warn(e.message);
    return false;
  } finally {
    window.Algebrite.run(`units(${prev})`);
  }
}

// Usage:
const inputArr = normalizeInputToArray(String.raw`@input`);

// draw(f, x, a, b, "options") -> <lia-chart> string, handed to the per-blo
  callback
// options: color (red, #f00), dashed/dotted/solid, width=n, fill, fill=a..
  ymin=, ymax=,
//          mark=x, point=x,y, vline=x, line=x1,y1,x2,y2 (repeatable)
window.liaChartDraw = function({ expr, variable, range, f, options, num, ca
  }) {
  const [a, b] = range ?? [-10, 10];
  const names = [].concat(expr);
  const palette = ["#5470c6", "#91cc75", "#fac858", "#ee6666", "#73c0de",
    "#3ba272", "#fc8452", "#9a60b4", "#ea7ccc"];
  const yAxis = { type: "value" };
  const y = (x, k) => { const v = [].concat(f(x))[k]; return Number.isFinit
    v : null; };

  const series = [];
  names.forEach((name, k) => {
    const style = { color: palette[k % palette.length], width: 2, type: "so
      ;
    const marks = [], lines = [];
    const opts = Array.isArray(options) ? options[k] : options;
    for (const token of (opts ?? "").split(/\s+/).filter(Boolean)) {
      const [key, val] = token.split("=");
      if (["solid", "dashed", "dotted"].includes(key)) style.type = key;
      else if (key === "width") style.width = num(val);
      else if (key === "fill") style.fill = val ? val.split("..").map(num)
        ;
      else if (key === "ymin") yAxis.min = num(val);
      else if (key === "ymax") yAxis.max = num(val);
      else if (key === "mark") marks.push({ coord: [num(val), y(num(val), k
      else if (key === "point") marks.push({ coord: val.split(",").map(num)
      else if (key === "vline") lines.push({ xAxis: num(val), lineStyle: {
        "dashed" } });
      else if (key === "line") {
        const [x1, y1, x2, y2] = val.split(" ").map(num);
```

```
      const [x1, y1, x2, y2] = val.split(",").map(num);
      lines.push([{ coord: [x1, y1], lineStyle: { type: "solid" } }, { co
        [x2, y2] }]);
    }
    else style.color = token;
  }

  const points = [];
  for (let i = 0; i <= 200; i++) points.push([a + (b - a) * i / 200, y(a
    a) * i / 200, k)]);

  // ponytail: pole heuristic (sign flip with a jump far beyond the typic
    ), adaptive sampling if it misfires
  const abs = points.map(p => Math.abs(p[1])).filter(Number.isFinite).sor
    v) => u - v);
  const typical = abs[Math.floor(abs.length * 0.9)] ?? 0;
  const data = [];
  points.forEach((p, i) => {
    const q = points[i - 1];
    if (q && q[1] !== null && p[1] !== null && q[1] * p[1] < 0 && Math.ab
      - q[1]) > 4 * typical) {
      data.push([(p[0] + q[0]) / 2, null]);
    }
    data.push(p);
  });

  const common = { name, type: "line", showSymbol: false, color: style.co
  series.push({ ...common, data, lineStyle: { width: style.width, type: s
    .type },
    areaStyle: style.fill === true ? { opacity: 0.3 } : undefined,
    markPoint: marks.length ? { symbol: "circle", symbolSize: 8, label: {
      false }, data: marks } : undefined,
    markLine: lines.length ? { symbol: "none", silent: true, label: { sho
      false },
      lineStyle: { color: style.color, width: 1.5 }, data: lines } : unde
        });

  if (Array.isArray(style.fill)) {
    const [lo, hi] = [Math.min(...style.fill), Math.max(...style.fill)];
    const area = [[lo, y(lo, k)], ...data.filter(([x]) => x > lo && x < h
      [hi, y(hi, k)]];
    series.push({ ...common, data: area, lineStyle: { width: 0 }, areaSty
      opacity: 0.3 } });
  }
});

const option = {
  backgroundColor: "transparent",
  legend: {},
  tooltip: { trigger: "axis" },
  xAxis: { type: "value", name: variable },
```

```
    yAxis,
    series
  };
  callback(`<lia-chart mode='dark' option='${JSON.stringify(option).replace
    "&#39;")}'></lia-chart>`);
}

// Runs Algebrite code line by line and prints every result into the given
// LiaScript console: as text, or as a formula after pretty(1) (pretty(0)
  switches
// back). Line by line keeps results and draw() charts in order.
// Returns the number of printed results.
window.algebritePretty = false;
window.algebriteRun = function(input, console, pretty) {
  window.Algebrite.setDrawHandler(window.liaChartDraw, html => console.html
    ));
  let printed = 0;
  input.split("\n").forEach((line, i) => {
    const toggle = line.match(/^\s*pretty\(([01])\)\s*((#|--).*)?$/);
    if (toggle) return window.algebritePretty = toggle[1] === "1";

    const [plain, latex] = window.Algebrite.run(line, true);
    if (!plain) return;
    printed++;
    if (plain.includes("Stop:")) {
      console.error(plain);
    } else if (pretty ?? window.algebritePretty) {
      const formula = latex.replace(/^\$\$|\$\$$/g, "").replace(/"/g, "&quo
      const tag = pretty ? `\\tag{${i + 1}}` : "";  // line numbers only in
        @Algebrite.pretty
      console.html(`<lia-formula displaymode="true" formula="${tag}${formul
        ></lia-formula>`);
    } else {
      console.log(plain);
    }
  });
  return printed;
}

@end

@Algebrite.pretty: <script> window.algebriteRun(String.raw`@input`, console
  ); "LIA: stop" </script>


@Algebrite.repl: <script>
  send.handle("input", input => {
    console.log("=> ");
    if (!window.algebriteRun(input, console)) console.log(input);
  });
```

```
  window.algebriteRun(String.raw`@input`, console);

  "LIA: terminal"
  </script>

@Algebrite.eval: <script> window.algebriteRun(String.raw`@input`, console);
  stop" </script>

@Algebrite.check: <script>
  const toList = s => {
    s = s.trim();
    return s.startsWith("[") && s.endsWith("]") ? s.slice(1, -1).split(";")
      => x.trim()) : [s];
  };
  const input = window.normalizeInputToArray(String.raw`@input`);
  const values = toList("@0");
  window.algebriteUnits(/units\s*=\s*1/.test("@1"), () =>
    input.length === values.length && input.every((item, i) => item !== ""
      window.Algebrite.run(window.inputClean(`(${window.latexToMath(item)})
        (${values[i]})`)) === "1"));
  </script>

@Algebrite.check_expression: <script>
  const input = window.normalizeInputToArray(String.raw`@input`)[0] ?? "";
  const toExpr = s => {
    const [lhs, rhs = "0"] = window.inputClean(s).split("=");
    return `(${lhs}) - (${rhs})`;
  };
  if (input.trim() === "") {
    send.lia("No input provided", [], false);
  } else {
    window.algebriteUnits(/units\s*=\s*1/.test("@1"), () =>
      window.Algebrite.run(`${toExpr(window.latexToMath(input))} == ${toExp
        )}`) === "1");
  }
  </script>


@Algebrite.check2: <script>
  const toList = s => {
    s = s.trim();
    return s.startsWith("[") && s.endsWith("]") ? s.slice(1, -1).split(";")
      => x.trim()) : [s];
  };
  const input = window.normalizeInputToArray(String.raw`@input`);
  const values = toList("@0");
  const tolerances = toList("@1");
  window.algebriteUnits(!/units\s*=\s*0/.test("@2"), () =>
    input.length === values.length && input.every((item, i) => item !== ""
      window.Algebrite.run(window.inputClean(`abs((${window.latexToMath(ite
        (${values[i]})) <= (${tolerances[i] ?? tolerances[0]})`)) === "1"))
```

```
  </script>


@Algebrite.check_margin: <script>
  window.algebriteUnits(!/units\s*=\s*0/.test("@2"), () => {
    const input = window.inputClean(window.latexToMath(window
      .normalizeInputToArray(String.raw`@input`)[0]));
    return window.Algebrite.run(`and((@0) <= (${input}), (${input}) <= (@1)
      === "1";
  });
  </script>
```

If you want to minimize loading effort in your LiaScript project, you can also copy this code and paste it into your main comment header, see the code in the raw file of this document.

https://raw.githubusercontent.com/liaTemplates/algebrite/master/README.md